
夜のZephyr RTOSハンズオン: STMicroelectronics Nucleo-C562REで学ぶZephyr入門

常田裕士

自己紹介

- 常田 裕士

- 2018年よりZephyrRTOSへのコミットを始める。
現在RaspberryPi Pico, etcのメンテナ/副メンテナを担当
- 本日は個人/ZephyrRTOSメンテナとしての講演になります。
- 著書/掲載記事等
 - 定番プリント基板設計 KiCad入門 (CQ出版社)
 - インターフェース 2026年 6月号/7月号 (CQ出版社)
AI/IoT時代のマイコン入門講座
- <https://www.slideshare.net/slideshow/kicad-53622272/53622272>
これ見てくれた方も居られるかも？

本日の流れ

Zephyrの概要

セットアップ(承前)

ボタンとLED - Zephyrの基礎

CAN

Zenoh

Zenoh+CAN

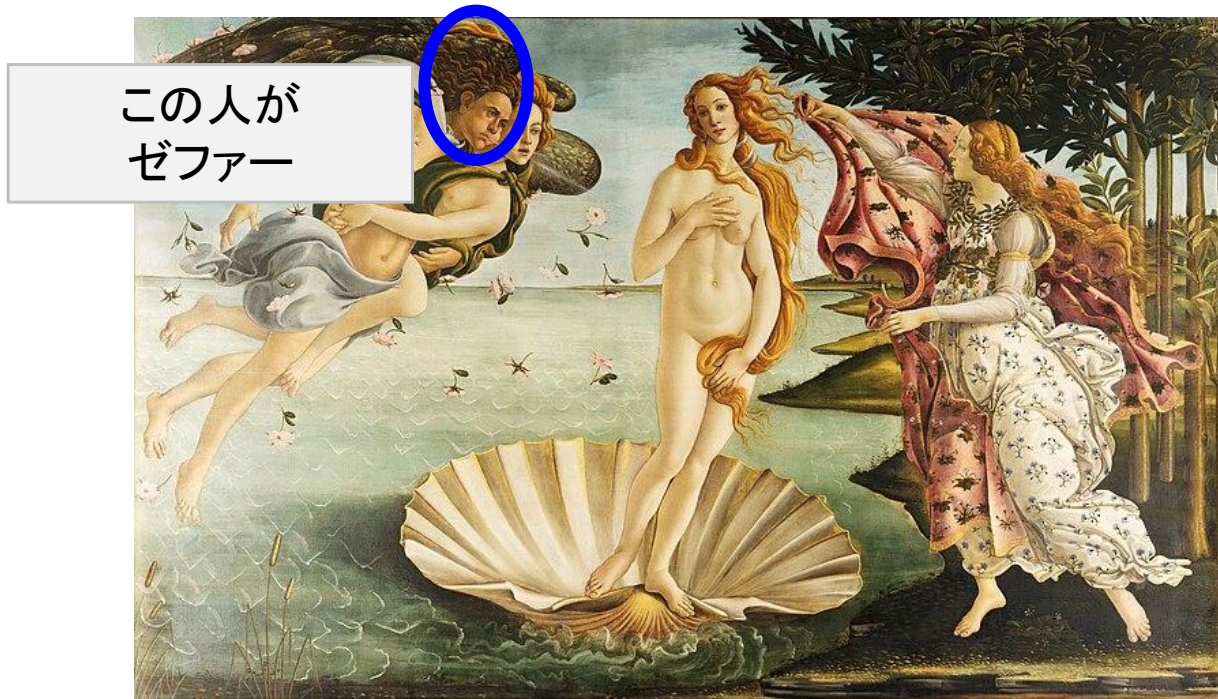
Zephyrの概要

Zephyrの特徴

- オープンソースのリアルタイムOS
 - ArduinoとかESP32など主に「Linuxが動かない」領域
- 軽量（メモリ/フットプリントが小さい）
- BluetoothやMatterなど、IoTの分野に強い
 - メカの制御にも使える
- POSIX対応をはじめとして多機能
- 対応しているSoC、開発ボードが非常に多い
- OSとしてHALを定義している



Zephyros (en:Zephyr)

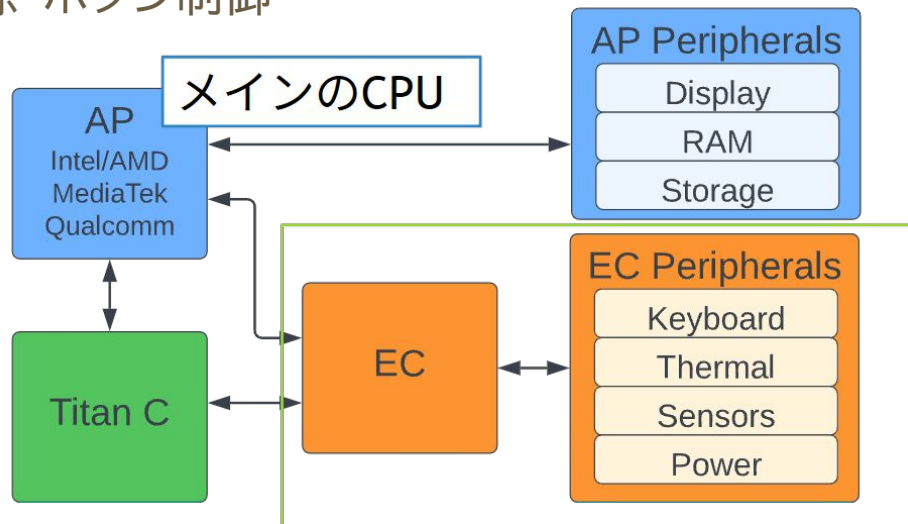


名前はギリシア神話の西風の神Zephyrosの英語読みから

誰がどこでZephyrを使ってる？ #1

- Google Chromebook の電源・ボタン制御

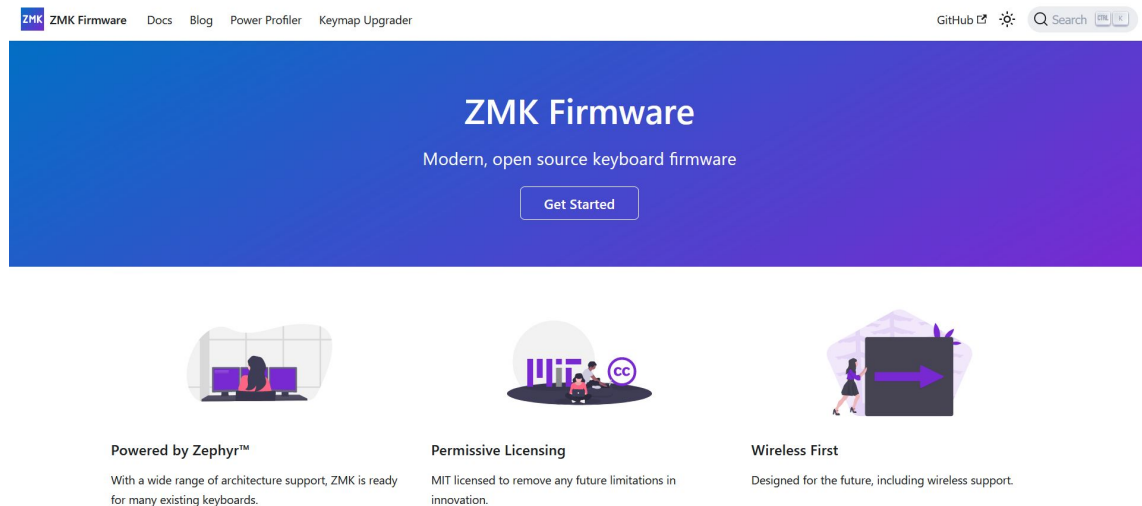
- 電源回りのハードウェアのインターフェースを統一して汎用的に使えるようにしている。
- メインのCPUでは動いていない。
- システムのスキマを埋めている



Zephyrでデバイスを制御している

誰がどこでZephyrを使ってる？ #2

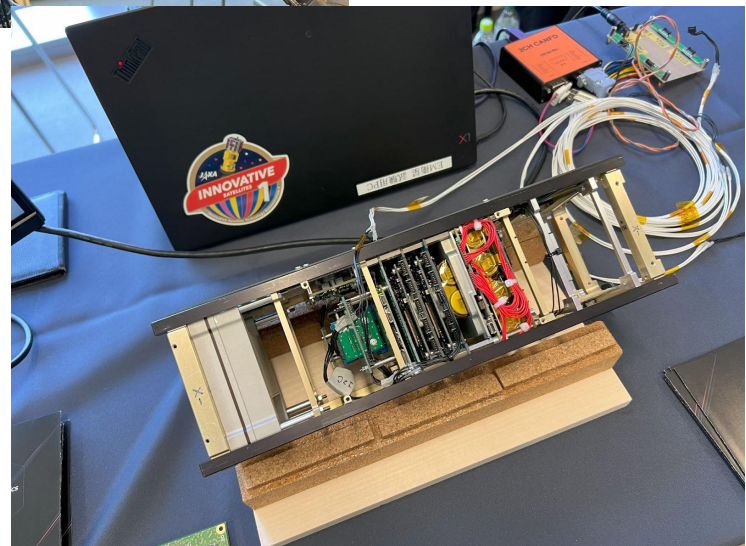
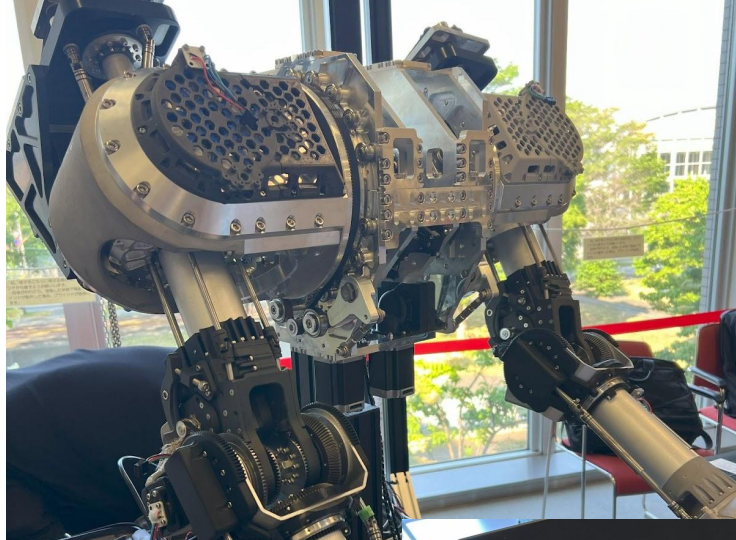
- ZMK



- カスタマイズ性に優れた、自作キーボード向けのファームウェア。
- Bluetoothのサポートに利用している。

誰がどこで Zephyrを使っ てる？ #3

- ロボット・宇宙開発
- 札幌のSpaceCubicsさんが、福島第一原発の廃炉ロボット、CubeSat(超小型人工衛星)への利用を行っている。
- ROS2と合わせて活用とのこと
- この方面だとZenohの通信プロトコルなども注目。



誰がどこでZephyrを使ってる？ #4

電子工作

最新のArduino Uno QではZephyrベースの
Arduinoコアが動作している。



ネットワーク(Connectivity)が強い

- Bluetooth
 - Bluetoothの「盟主」のNordic Semiconductorが精力的にコミット。
- TCP/IP
 - OSにインテグレートされて「第1級」のサポートとなっている点が優位。
- Matter/OpenThread
 - スマートホームもBluetooth同様手厚い。
- ModBus
 - FA系も通信規格もスコープに入っている。
- CANBUS
 - もちろん車載もある。

まとめ

- Zephyrとは
 - ネットワーク(Connectivity), IoTに強いオープンソースのリアルタイムOS。
 - 電子工作から宇宙開発まで広い用途で使われている。
 - 移植性が高く、STMicroelectronics製品をはじめとして多数のボードで動作する。
 - 機能安全認証は目下取組中。

STMicroelectronics
Nucleo-C562RE

ボード紹介

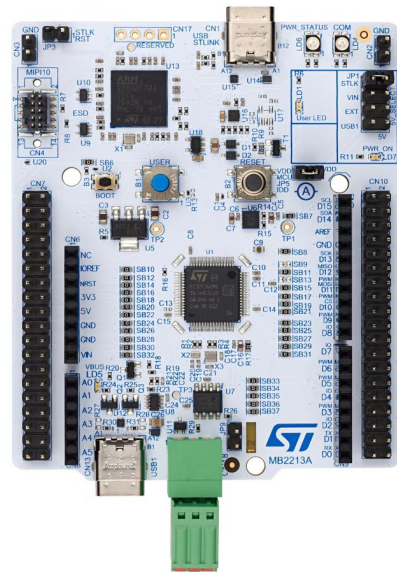
STMicroelectronics 様より、本セッションのために
Nucleo-C562RE をご提供いただきました。

Cortex-M33/512KB Flash/128KB SRAM

CANトランシーバー搭載で緑色の端子が特徴的。

トランシーバーの接続は意外と「ハマる」ポイント。

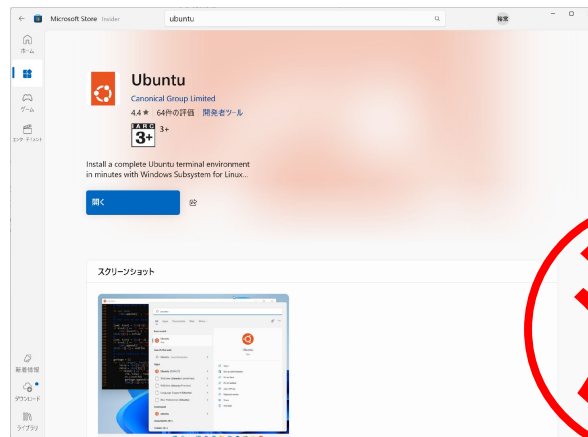
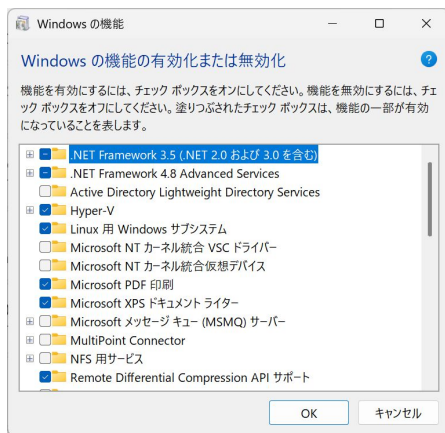
「車載やロボティクスで使いやすい、
こなれたベーシックモデル」



セッティングアップ(承前)

環境設定(Windows) #1

- Windowsの場合はWSLを使うのが便利。
 - コントロールパネル→プログラム→Windowsの機能の有効化または無効化
 - Windows StoreよりUbuntuをインストール



環境設定(Windows) #2

- WSL USB Manager
 - <https://gitlab.com/alelec/wsl-usb-gui>
- USBデバイスのWSL環境への接続・切断を行う。
- コマンドラインで行ってもいいが、これがあると敷居がかなり低くなる



環境のセットアップ

- 素直にGetting Started Guide

[https://docs.zephyrproject.org/latest/develop/getting started/index.html](https://docs.zephyrproject.org/latest/develop/getting%20started/index.html)

に乗っかってaptの更新

```
sudo apt update
sudo apt -y upgrade
sudo apt install --no-install-recommends git cmake ninja-build gperf \
  ccache dfu-util device-tree-compiler wget python3-dev python3-venv \
  python3-tk xz-utils file make gcc gcc-multilib g++-multilib \
  libstdc++2-dev libmagic1
```

済

Python仮想環境のセットアップ

今回は講座向けのリポジトリの設定を使って環境をセットアップする

```
mkdir swest28 ; cd swest28
python3 -m venv ~/zephyrproject/.venv # venvを作成
source ~/zephyrproject/.venv/bin/activate # アクティベート
pip install west # Zephyrの万能ナイフのwestコマンドをインストール
git clone https://github.com/soburi/s1c\_zephyr\_at\_night # 今回の教材
west init -l s1c_zephyr_at_night # zephyrのリポジトリをcloneする
west update # DL量を減らす場合はmanifestファイルを用意する
west zephyr-export # Cmakeの設定
# pythonのモジュールのインストール
west packages pip --install
```

済

SDK(コンパイラ, etc)をインストール

- 引き続き、指示通りに

```
cd ~/zephyrproject/zephyr  
west sdk install # ARMだけなら -t arm-zephyr-eabi を指定する
```

- このコマンド(west sdk)は私が作りました！
- これで、環境セットアップはおわり。



ボタンとLED: Zephyrの基礎

コンソールを開く

コンソールでPythonの仮想環境に入る。
この後、コンソールを開く操作がある場合、はじめにこのコマンドを実行する

Ubuntu

```
cd swest28  
. .venv/bin/activate  
cd slc_zephyr_at_night
```

Windows

```
cd swest28  
.venv/Scripts/activate.bat  
cd slc_zephyr_at_night
```

本講座向けオフライン設定用のUSBを使った場合はそちらの指示に従うこと

Button サンプル をビルドする

- 定番は hello worldかblinkyだが、今回は時間ないのでこちらで。
- 以下のコマンドでビルドする

```
west build -p -b nucleo_c562re button
```

- 赤字のボード指定子の詳細は
https://docs.zephyrproject.org/latest/hardware/porting/board_porting.html#board-terminology
を参照

Button サンプル を書き込む

以下のコマンドで書き込みを行う。

```
west flash --runner pyocd
```

- --runner で書き込みに使うプログラム(on chip debugger)を指定する。
- PyOCDはPythonで書かれたARM向けのデバッグツール。
- -rと省略可能。

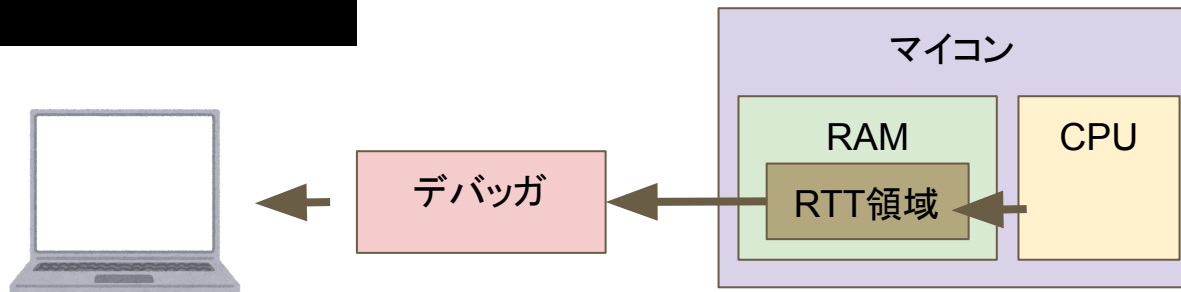
RTT

通常、ログなどの出力にはUART(シリアル)に出力して、Tera Term/screenコマンドなどで確認するが、今回は別の目的で使うので使えない。

RTT (RealTime Transfer)はRAMの一部にログ用のバッファを用意して、デバッガがログを吸い上げて表示する。

以下のコマンドでRTTのログを表示する

```
west rtt --runner pyocd
```



デバッグする

- デバッグもwestコマンドを叩くと**コンソールの**gdb が立ち上がる

```
west debug -r pyocd
```

- gdbの簡単なコマンド
 - b: breakpoint を設定、b mainのようにしてみる。
 - c: プログラムの実行を継続
 - s: ステップ実行
 - n: 次の行まで実行

west command

- Zephyrの総合的なフロントエンドを提供するコマンド
 - Zephyrの操作はほぼすべてこのコマンド経由で実行する
 - Zephyrが西風の神様なのでこの名前。twisterなんてのも居る。
 - 以前はcmake & ninjaを直接使っていたが、全部west経由に。
 - いまではZephyrの特徴の一つと言えるほどプロセスの中核に居座っている
- westコマンドの操作はすべてのボードで共通。
 - 開発のノウハウも他のボードに移行可能
- GitHubのghコマンドやVueやflutterのコマンドなど、Web系の文脈から見ると自然な導入に見える
 - 組み込みの文脈だけがZephyrのルーツではない！

プログラムの構成と DeviceTree

プログラムの構成

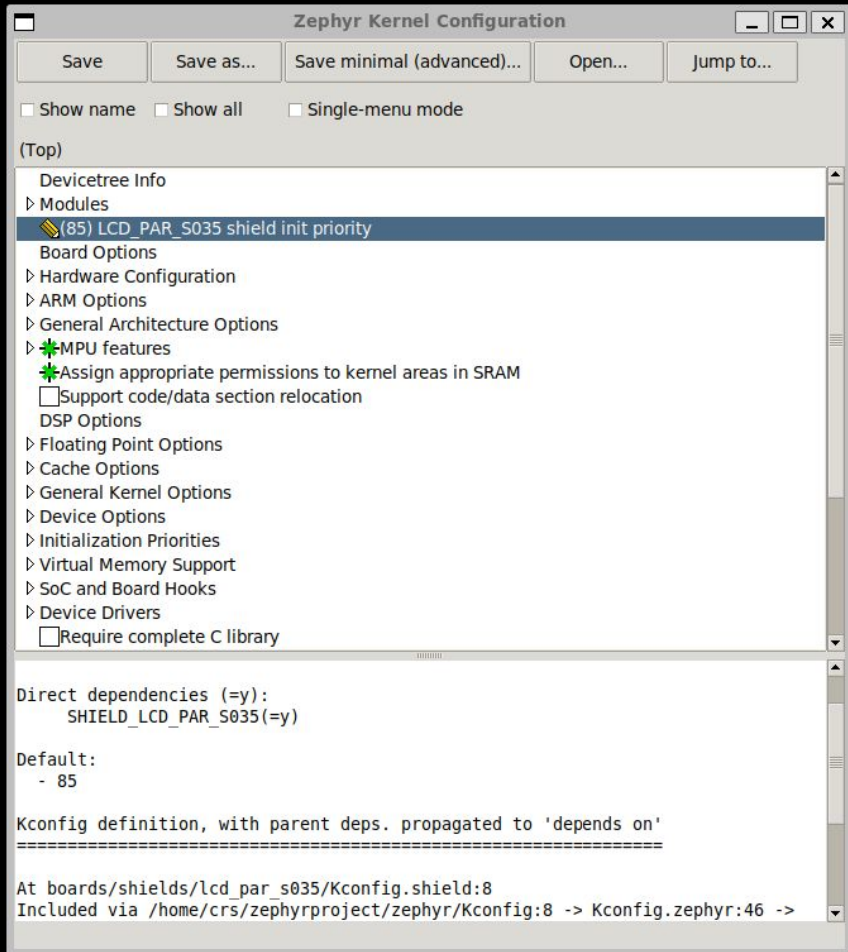
- ファイル構成:
 - **CMakeLists.txt**: CMakeの設定ファイル。コンパイル対象の設定に使う。
 - **prj.conf**: ビルドオプションの設定
 - **src/main.c**: これが本体
 - **README.rst**:
 - 単なるドキュメント。ZephyrではreStructuredTextが標準

CMakeLists.txt

- CMakeの設定ファイルは基本的にはコンパイル対象とincludeパスの追加のみ
- `find_package(Zephyr REQUIRED HINTS $ENV{ZEPHYR_BASE})`はオマジナイとして必須。
 - Zephyr固有のCMakeの設定がまるっと読み込まれる。
- **直接cmakeコマンドは滅多に使わないので、ここで頑張らないこと。**
- だいたいの場合、サンプルのコピペで十分。

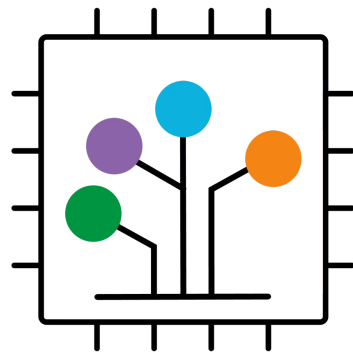
prj.conf

- 設定のメインはこれで行う。
- **LinuxのKconfig**の移植。
- CONFIG_~=y で各種オプションを設定する。
- オプションは山程ある。(現状20000程度)
- west build -t menuconfigで設定ができる。
- west build -t guiconfig でGUIの画面で設定ができる。
- 今回のサンプルではRTTの設定をここで行っている。



DeviceTree

- DeviceTreeはZephyrを特徴づける最大の要素
 - 「汎用RTOS」であるため、カーネルの基本機能で強い「色」はついていない。
- Linuxと違って、DeviceTreeを**静的にコンパイル**して組み込む。
- 組み込み用途に必要なフットプリント最小化と、実装と構成の分離を同時に実現している。
 - このトレードとして、**大変煩雑なC言語のマクロの記述が必要になる**
 - このpros/consのトレードオフの判断こそがZephyrを特徴づけている



Linuxでの DeviceTree

バイナリファイルを使う

- DeviceTreeの仕様にある、DTB形式(DeviceTree Blob, バイナリ形式)を使う。
- テキストで記述されたDTS(DeviceTree Source)をDTBにコンパイルする
- カーネルが起動時にDTBを読んで、構成(Configuration)データとして利用する。

ZephyrでのDeviceTree

- DeviceTreeの書式はLinuxと同じ(DeviceTreeの仕様通り)
- ZephyrではDTBを使わない
- DeviceTreeから巨大なC言語のコード(マクロ)を生成する
- 動的な構成の変更はできない。
- **実装と構成を分離**して、コードの再利用性を高めるのが目的。
 - Linuxでもこの目的で導入された。
[LKML: Linus Torvalds: Re: \[GIT PULL\] omap changes for v2.6.39 merge window](#)
(ARM関連のコードが激増し、Linus氏が激怒したという逸話)
 - Zephyrでも同様に効果が上がっている。

具体的な現れ方

- buttonのソースでDeviceTreeの情報にアクセスしているのは以下の箇所。

```
#define SW0_NODE          DT_ALIAS(sw0)

static const struct gpio_dt_spec button =
GPIO_DT_SPEC_GET_OR(SW0_NODE, gpios, {0});

static struct gpio_dt_spec led =
GPIO_DT_SPEC_GET_OR(DT_ALIAS(led0), gpios, {0});
```

具体的なGPIOのハードの構成をC言語の実装のソースコードから追い出すことが出来る。
これがないと #ifdef 地獄になってしまう...

- 以下のDeviceTree定義と対応する。

```
/ {
    aliases {
        led0 = &green_led_1;
        sw0 = &user_button;
    };

    gpio_keys {
        compatible = "gpio-keys";

        user_button: button {
            label = "USER";
            gpios = <&gpio0 13 GPIO_ACTIVE_HIGH>;
            zephyr,code = <INPUT_KEY_0>;
        };
    };

    leds: leds {
        compatible = "gpio-leds";

        green_led_1: led_1 {
            /* LD1 may conflict with SPI1 SCK on PA5 */
            gpios = <&gpioa 5 GPIO_ACTIVE_HIGH>;
            label = "LD1";
        };
    };
};
```

CAN

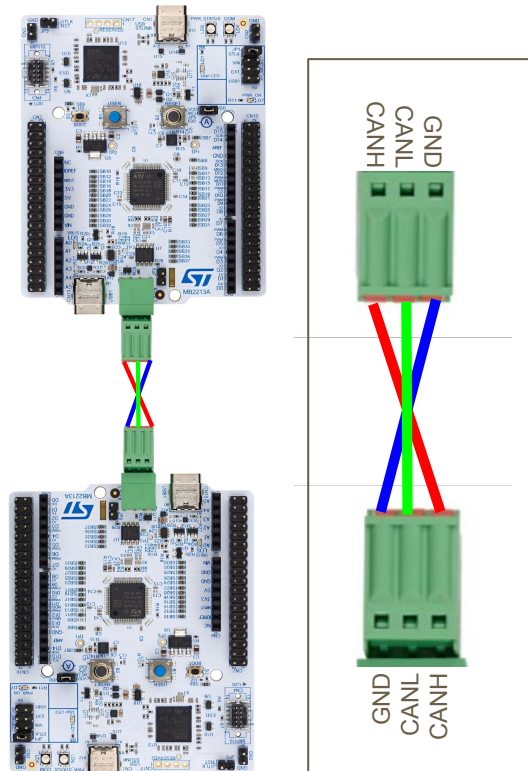
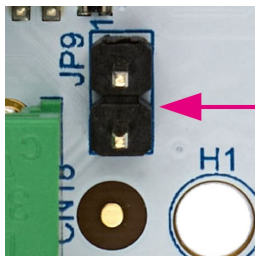
CANをつなぐ

隣の人とやりましょう。

ジャンプワイヤで緑色のCANコネクタを接続する。

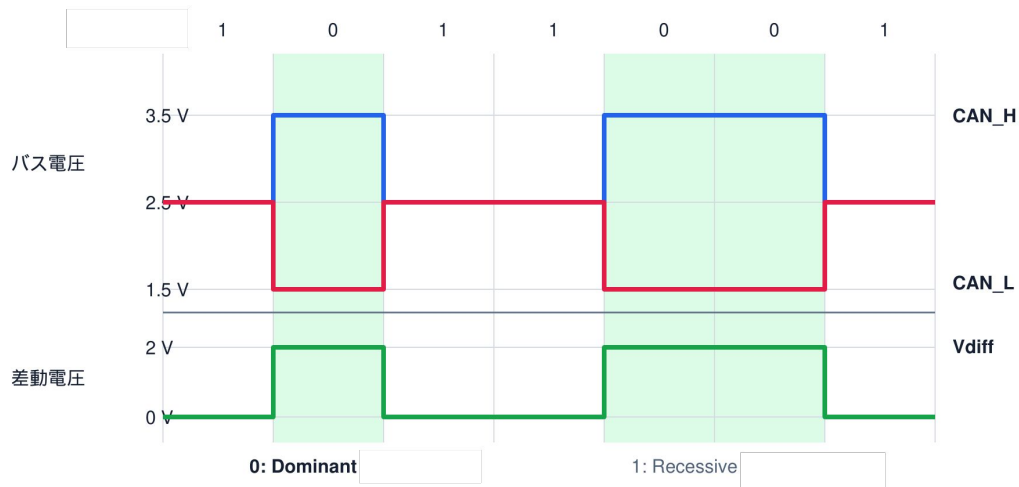
RX-TXのような交差はなし。同じ名前の端子同士を接続する。

JP9のピンヘッダにジャンパピンを差してターミネーターを有効にする。



CAN通信 #1

- 電氣的にはUSB, HDMI, Ethernetと同じ差動ペア。
- 2本のケーブルに+/-反転させた同じ信号を流して通信する。
- ノイズに強い。



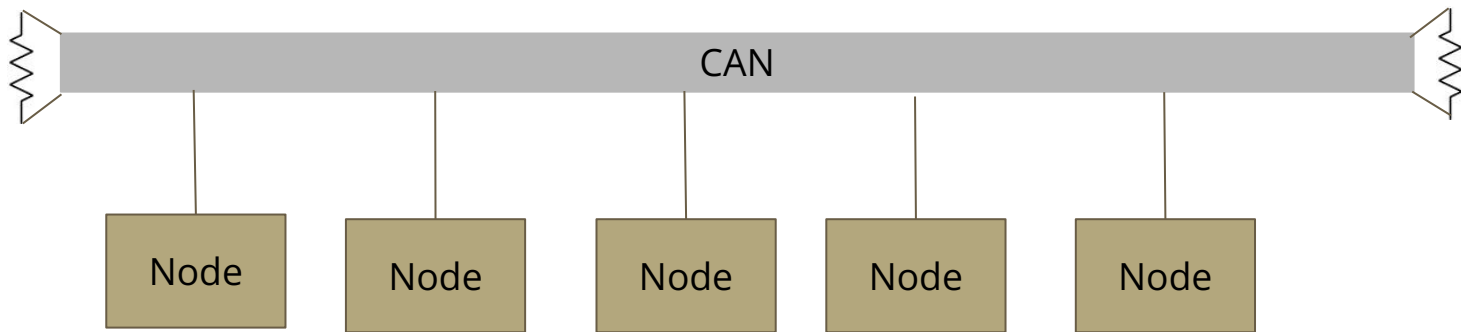
CAN通信 #2

バス型のトポロジで、数珠繋ぎでたくさんのデバイスをつなげる。

両端にターミネータの抵抗が必要。

無印のCAN(～1Mbps)とより高速なCANFD(～8Mbps)がある。

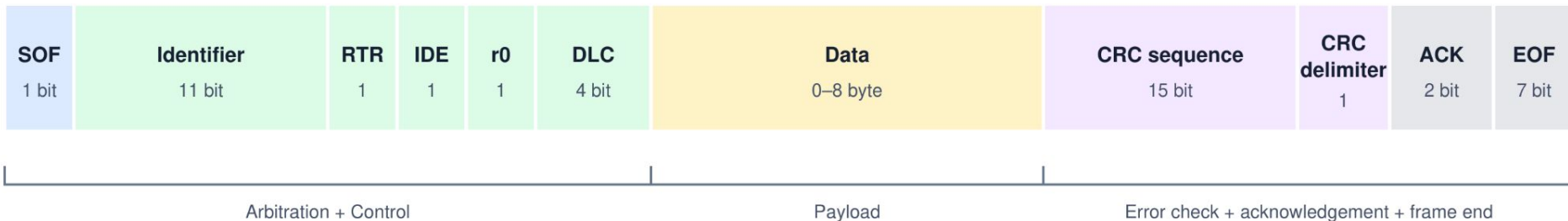
Nucleo-C562REはCANFD搭載。



CAN通信 #3

データフレーム単位で通信を行う。

データフレームにはデータ種別を表すための11ビットのIDを振る。



CAN通信プログラム

以下のコマンドでサンプルをビルドする。

```
west build -p -b nucleo_c562re can_comm
```

書き込み、ログの表示はbuttonの時と同様に操作する。

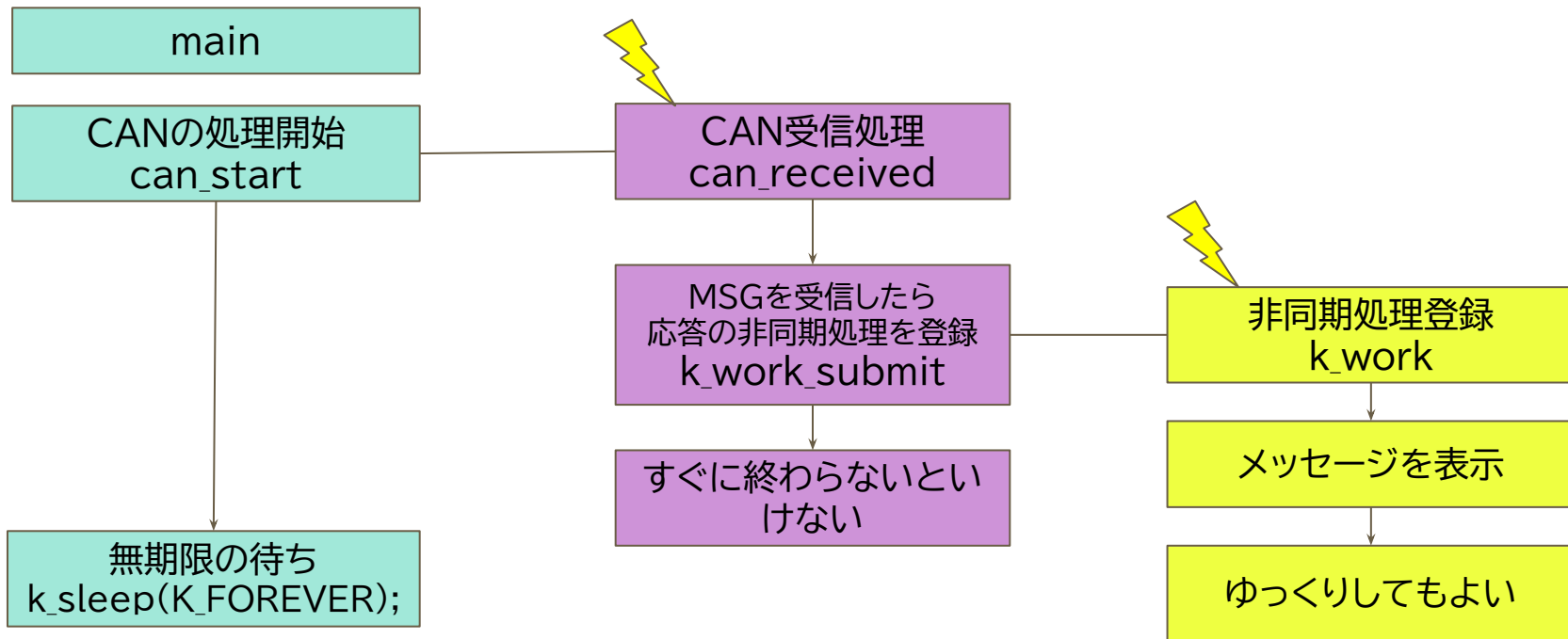
ボタンを押すとCANメッセージが送信される。

受信したメッセージのIDをログに出力する。

→ ボタンを押して、RTTで相手方のマイコンのログを確認する。

処理の流れ

CAN受信の割込みの中で時間のかかる処理をやってはいけないので非同期化



CANプログラム： 演習

配布した番号をCAN_TARGET_ID_SELFに、
相手方の人の番号をCAN_TARGET_ID_TARGETに設定して
実行してみよう。

Zenoh

Zenoh

Zenohはpub/sub型の通信プロトコル。

文字列で示されるトピック(key expression)にsubscribeすると、このトピックに関するデータが通知される。

ルーターがあるので大規模なネットワークを構築できる。

プロトコルヘッダがとても小さい。

Zenoh サンプル #1

ビルドと書き込みはこれまでと同様に行う。

```
west build -p -b nucleo_c562re zenoh_serial
```

Zenoh サンプル #2

別のコンソールを開いて、zenohdを起動する。

```
tools/zenohd -l 'serial//dev/ttyACM0#baudrate=115200' -l  
'tcp/127.0.0.1:7447'
```

赤字/下線の箇所はnucleoをつないでいるシリアルポートを指定する。

起動したらnucleoのリセットボタンを押す。

nucleoのボタンを押すと、zenohdにメッセージが流れる。

Zenoh サンプル #3

さらに別のコンソールを開いて、

```
python3 tools/z_sub.py
```

を実行する。nucleoのボタンを押すところにnucleoのプログラムがpublishしたトピックが表示される。

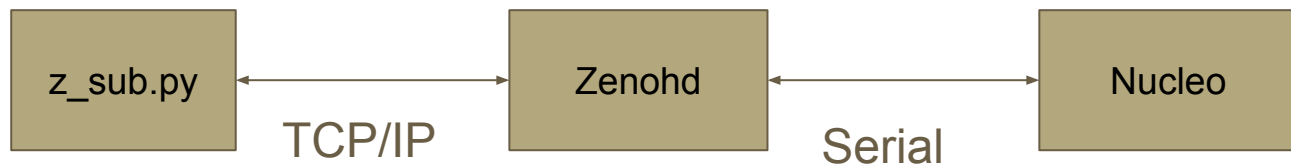
Zenoh サンプル続き

さらに別のコンソールを開いて、

```
python3 tools/z_pub.py
```

メッセージがzenohdを介して送られ、nucleoがsubscribeしているトピックが表示される。

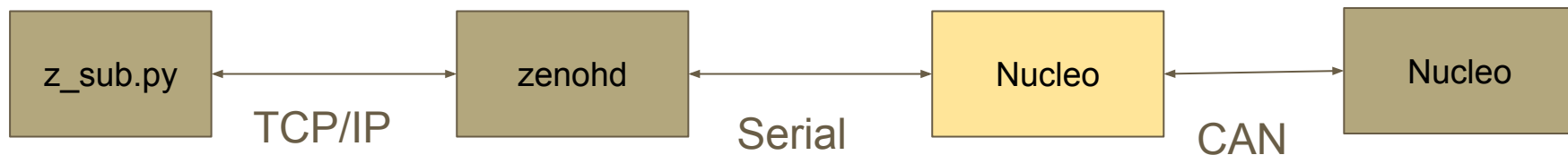
ルーティング



zenohdがルーターとして
メッセージを中継・配送し
ている

Zenoh+CAN

Zenoh-CAN gatewayにしてみる。



1フレーム64バイトのCANFD通信にZenohを乗せるのはつらいので、CANメッセージをマップしてZenohで読めるようにする

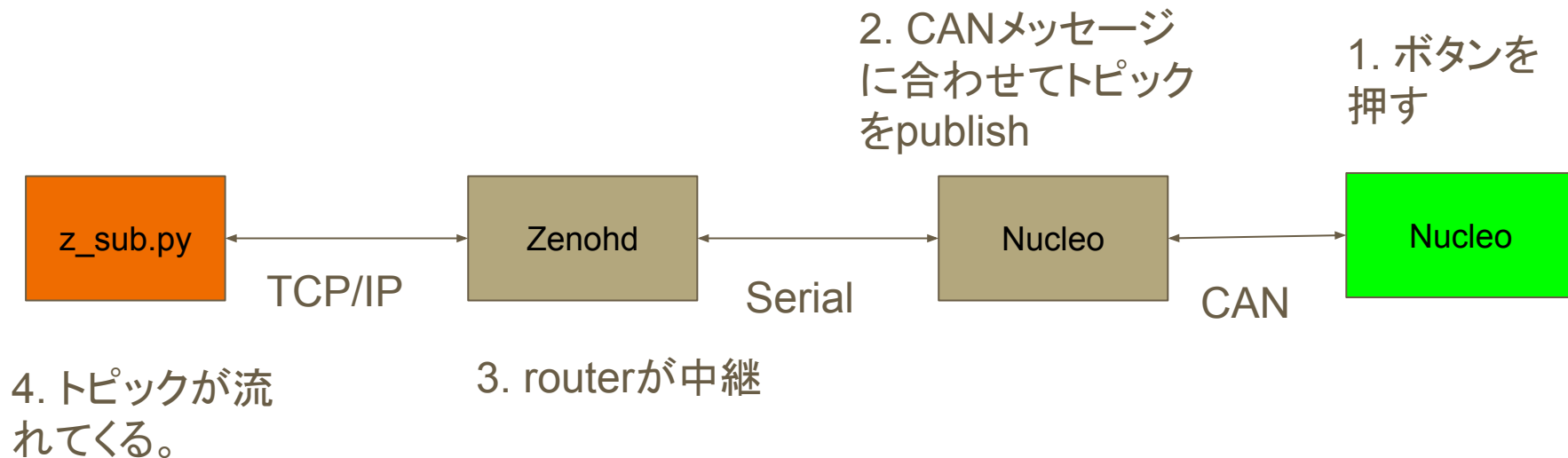
マッピング仕様

ここでは話を簡単にするために、CANメッセージのIDだけを扱うことにしてデータの中身は無視することにする。

(データの扱いは皆様の宿題としてお持ち帰りください。)

- CANメッセージを受信したとき、“can/[id]/rx” のトピックをpublishする。
- “can/[id]/tx” がpublishされたら ID:[id]のCANメッセージを送信する。
- [id]はCANメッセージのIDを3桁の16進数で表したもの

メッセージの流れ



Zenoh+CAN サンプル

内容としてはこれまでの2つのサンプルを統合したもの。
ID-トピックの変換処理だけ追加。

CANの時の演習と同様、
配布した番号をCAN_TARGET_ID_SELFに、
相手方の人の番号をCAN_TARGET_ID_TARGETに設定する。

ビルドは

```
west build -p -b nucleo_c562re can_zenoh
```

対向のNucleoでボタンを押すとトピックが流れてくる。

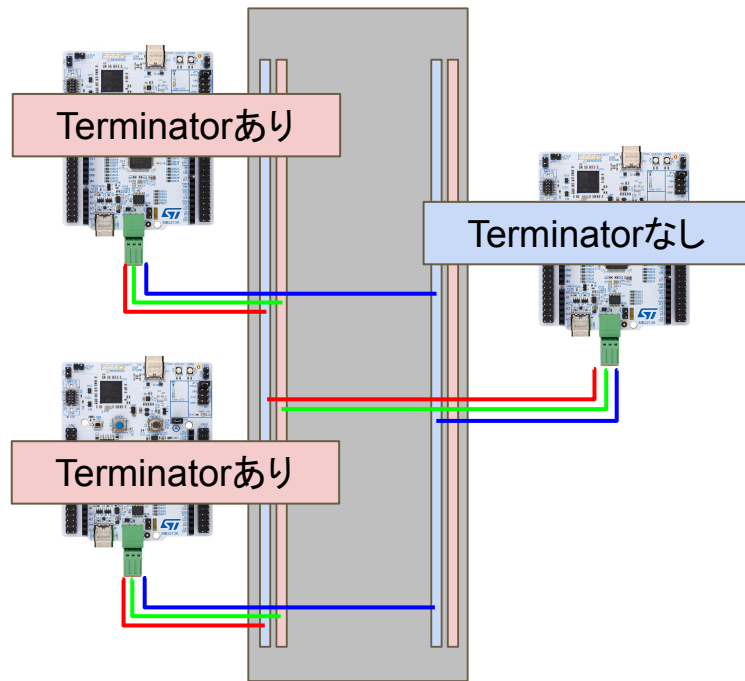
CANにたくさんNucleoをつなぐ

ブレッドボードの電源ラインの部分を使って、複数台のNucleoを同一のCANネットワークに接続する。

バスの端にあるNucleoはターミネータ(JP9)を有効にする。

近くの人と集まってつないでみよう。

ブレッドボード同士をつなぐともっとたくさんつながる？！



車とECU

- 自動車の中にはECU (Electronic Control Unit)としてマイコンが大量に入っている。
- 1台の車で100台以上のECUがあると言われている。
- それを接続するケーブルの総延長は5kmに達するとともに。
- 車載のネットワークも今回試したCANのネットワークの発展的なもの。

Recap

まとめ

- Zephyrの概要を理解した
 - オープンソースのRTOS
 - 色々なボードがすべてwestコマンドによる同一の操作で開発できる。
 - Linuxから継承したDeviceTreeなどの仕組みで多くのデバイスがサポートされている。
- CAN通信の基礎を理解した
 - 車載やロボティクスで使われる差動信号による通信
 - バス型のネットワークで複数台の通信ができる。端のノードにはターミネータが必要
 - 短いデータフレームで通信し、それぞれのデータフレームには対象機能を示すIDが付けられる。
- Zenohを触ってみた
 - Pub/Sub型の通信プロトコル。

ZenohはPub/Subの通信プロトコル。ルーターがあり、大きなネットワークが作れる。

CANにそのままZenohの通信がないのでゲートウェイが必要。